

LoggerAppsMsg()

Synopsis

```
int  LoggerAppsMsg(devProd, message_group, severity, message_string, ... )
int  devProd;
int  message_group;
int  severity;
char *message_string;
```

Description

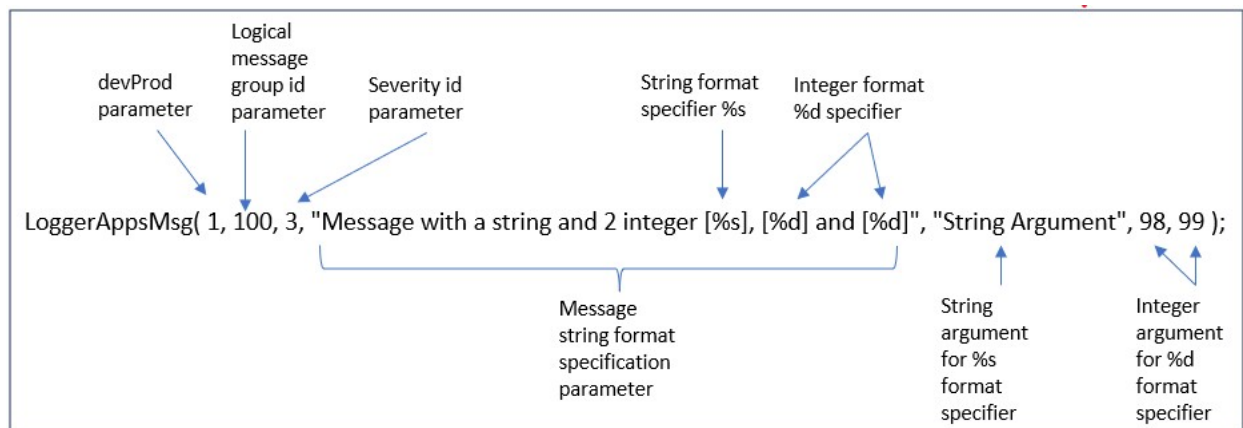
LoggerAppsMsg() decomposes argument parameters into id=value pairs, validates for errors, encode to reduce storage capacity, and securely protects to ensure confidentiality, integrity, and authenticity.¹

Example Usage

```
LoggerAppsMsg(1,100,3,"Message that generates 1 string and 2 integers [%s],
[%d] and [%d]","String Argument", 98,99);
```

Generates formatted output log message:

```
02/09/2026 12:01:55 [3650:sample_program] [DATABASE] [INFORM] Message that
generates 1 string and 2 integers [String Argument], [98] and [99]
```



The devProd integer parameter must be 1 or 2, where 1 means generate unprotected log messages.²

The logical message_group integer parameter is the id part of the id=value pairs defined in file LoggerMessageDefines.txt. The id part of 100 is translated and mapped to the corresponding value part of DATABASE in the output message. [Refer to the LoggerMessageDefines.txt document](#) for further details.

The severity integer parameter is the id part of the id=value pairs defined in file LoggerMessageDefines.txt. The id part of 3 is translated and mapped to the corresponding value part of INFORM in the output message. [Refer to the LoggerMessageDefines.txt document](#) for further details.

The message_string format string specification parameter is a C-Programming Language printf() style format string specification that comprises of format string text and format specifiers.

¹ The Free Version supports decompose and validate format string specification and string argument parameters.

² The Free Version supports devProd option 1 only.

The reminder of the `LoggerAppsMsg()` parameters (as indicated by "...") are the corresponding format string specifiers argument values for `message_string`.

NOTE: Refer to the [Wandi-SASL Capabilities section Adjustable Parameters](#) for validation guidelines used by `LoggerAppsMsg()` to check and validate parameters.

`LoggerAppsMsg()` returns 0 on success and -1 on error.

This API function is failed safe which means when an error occurs the application can continue processing.

Error Codes

Error codes are written to the following files, where the `<cpid>` is the application process id number:

- `/tmp/<cpid>-LoggerAppsMessage.txt`
- `$LOGGER_HOME_PATH/LoggerOutput/<cpid>-LoggerAppsMessage.txt`

135	Invalid log env <code>LOGGER_HOME_PATH</code> parameter.
136	Invalid logical message grouping and/or severity id parameter.
137	Invalid format message string text.
138	Too many format string specifiers.
139	String argument pointer parameter is NULL.
140	String argument pointer is negative.
141	String argument not properly NULL terminated.
142	String argument exceeded allowed length.
143	Invalid string argument.
1018	Invalid format string specifier type.
115, 116, 117	Invalid message text format string.
104, 106	Invalid character in message text string or string argument.
105	String argument length is zero.